

Data Structure and Algorithm

COURSE CODE: CS-201

TOPIC: REVISION OF STRUCTURES & POINTERS

COURSE INSTRUCTOR: HABIBA ARSHAD



Goals for today

To introduce :

- Structures
- Pointers
- Dynamic Memory allocation

Structures

- As array is the collection of same type of elements but it may be possible to take a different type of elements together. It is useful to group different types of data which has some meaningful information.
- If we want to create a record of any person which contains the name, age and address of that person then we make the structure of the record.

Structures

A structure is a collection of simple variables.

The variables in a structure can be of different types: Some can be int, some can be float, and so on.

The data items in a structure are called the *members* of the structure.

Defining a Structure

The structure definition tells the compiler about the detail of the structure and how the structure is organized

- It specifies what members the structure will have.

A structure is declared using the keyword **struct**.

It does not set aside any space in memory or even name any variables.

- This is unlike the definition of a simple variable, which does set aside memory.

The structure declaration is also called structure specifier.

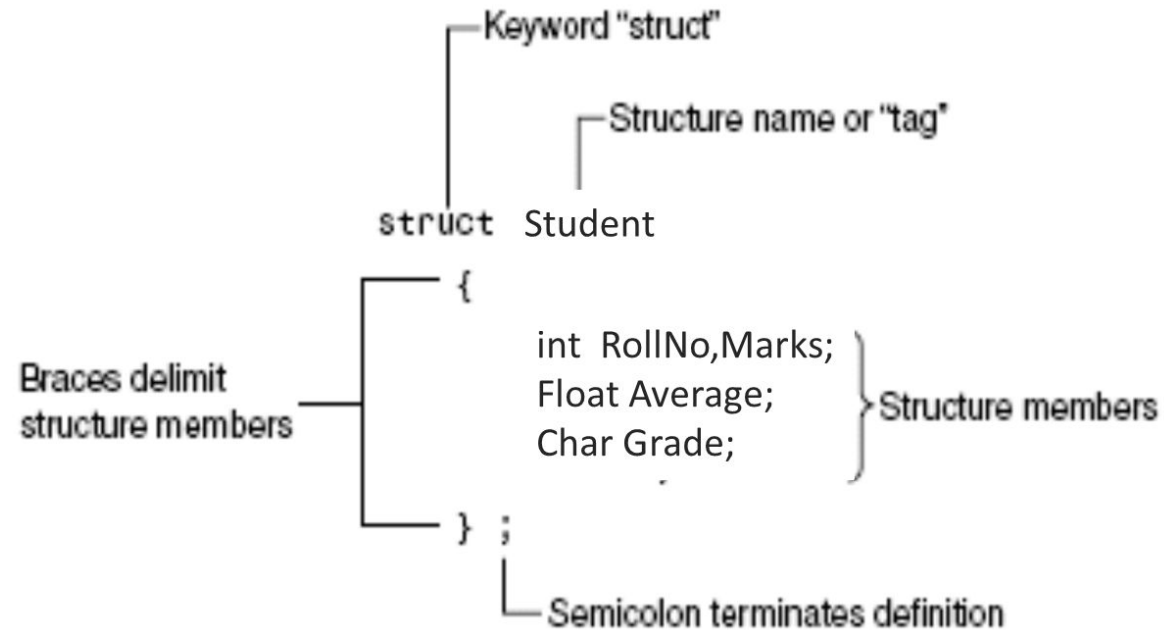
Defining a Structure

The structure definition tells how the structure is organized: It specifies what members the structure will have.

Syntax:

```
struct structure_name
{
    DataType Identifier_a;
    DataType Identifier_b;
    DataType Identifier_c;
};
```

Defining a Structures



- The above structure contains four member variables.
- A new data type student is created in the above declaration.
- A variable of type student can store four variables at one time.

Declaring a Structure Variable

- The structure variable can be declared after the defining a structure.
- The process of declaring a structure variable is same as declaring a variable of basic types such as int and char.
- The declaration tells the compiler to allocate memory space for the variable.
- The compiler automatically allocates sufficient memory according to the elements of the structure.
- The structure can be defined inside or outside the main() function
- If the structure is defined outside main() its variable can be declared inside or outside the main()
- If the structure is defined inside main() its variable can only be declared inside the main()

Declaring a Structure Variable

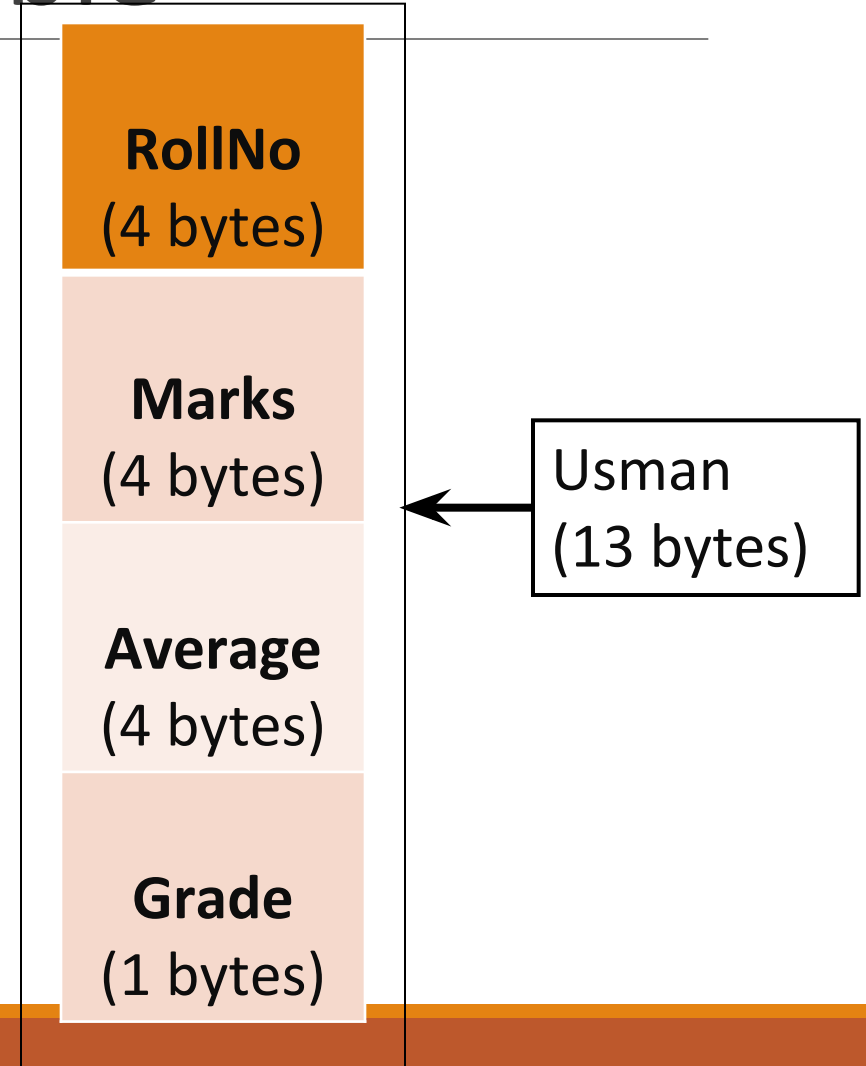
Syntax

Struct_Name Identifier;

Example

```
struct Student
{
    int RollNo, Marks;
    float Average;
    char Grade;
};
```

```
Student Usman;
```



Accessing Structure Members

Once a structure variable has been defined, its members can be accessed using something called the ***dot operator***.

- The structure member is written in three parts:
- the name of the structure variable on the left side of the dot
- The dot operator
- and the member variable name on the right side of the dot
- The dot operator is also called member access operator

Syntax

```
Struct_Var . Member_Variable;
```

Accessing Structure Members

Student Usman;

Usman.RollNo = 10;

Usman.Marks = 864;

Usman.Average = 89.52;

Usman.Grade = 'A';

Array Type Member of Structure

- Members of the structure can be of different types
- Can be simple variable, or array variables

Example

```
struct Student
{
    int RollNo;
    char Name[16];
    int Marks[4];
};
```

```
Student s = { 10 , "Usman" , {35, 45, 25, 22}};
```

Pointers

A pointer is a variable which holds the address of other variable of the specified data type (like int, float, char).

In programming we basically use pointers to store the other variable's address.

A pointer variable is declared with a '*' before it.

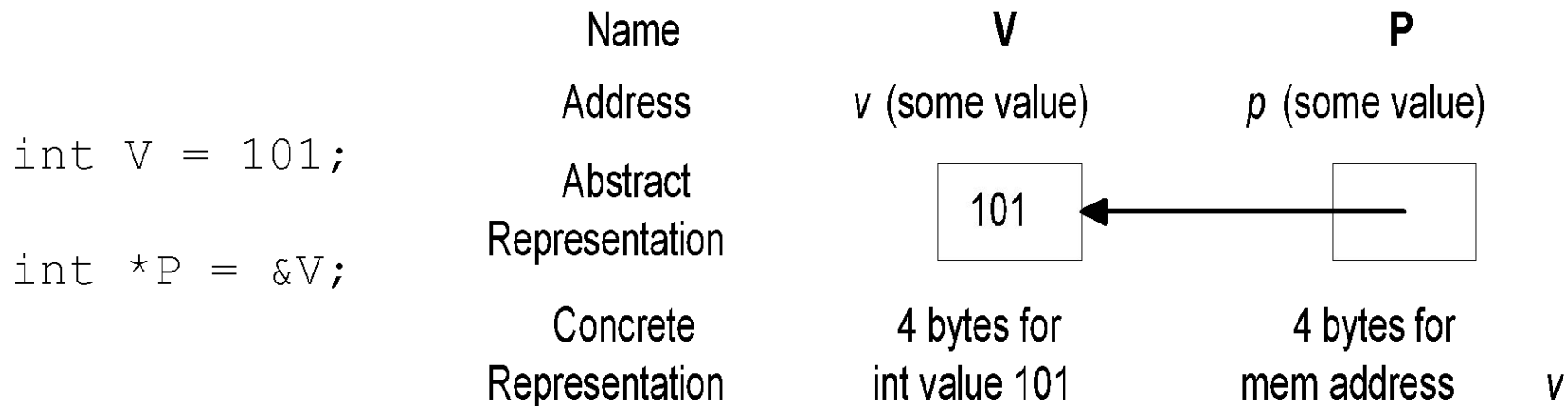
It contains the memory address (locations) of another variable which is declared as the pointer type.

Pointer Basics

Variables are allocated at *addresses* in computer memory (address depends on computer/operating system)

Name of the variable is a reference to that memory address

A pointer variable contains a representation of an address of another variable (P is a pointer variable in the following):



Indirection (*) Operator

A pointer variable contains a memory address

To refer to the *contents* of the variable that the pointer points to, we use indirection operator

Syntax: **PointerVariable*

Example:

```
int V = 101;
```

```
int *P = &V;
```

```
/* Then *P would refer to the contents of the variable V (in this case, the  
integer 101) */
```

Pointer Variable Definition

Basic syntax: *Type *Name*

Examples:

```
int *P;    /* P is var that can point to an int var */
```

```
float *Q; /* Q is a float pointer */
```

```
char *R; /* R is a char pointer */
```

Complex example:

```
int *AP[5]; /* AP is an array of 5 pointers to ints */
```

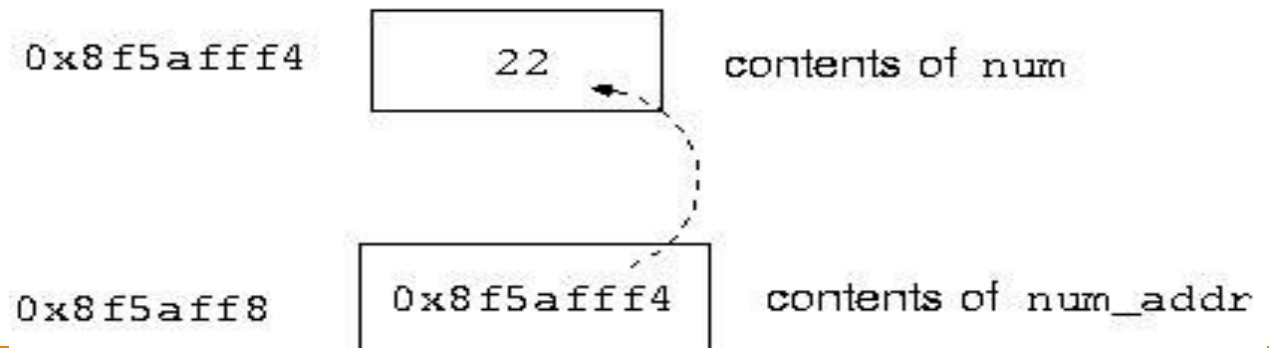

The & Operator

- The & is the address operator. It represents the address of the variable.
- Storing address in pointers

```
int num = 22;
```

```
int * num_addr = &num;
```

```
num = 22;  
num_addr = &num
```



Point to be noted

`int *a=&b` is an easy way of assigning the address of 'b' to pointer 'a'. This doesn't mean *a is equal to 'address of b'.

The size of the pointer variable varies according to the size of the computer architecture i.e. a 32-bit operating system takes 4 bytes while a 64-bit takes 8 bytes.

-> is the member access operator in case of pointers.

Memory Management with Pointers

The process of allocating and de-allocating memory is known as memory management.

Static Memory Allocation

Memory is allocated at compilation time

Dynamic Memory

Memory is allocated at running time

Static memory allocation

Memory to be allocated is specified while writing programs

```
int i , j , k ;  
char name[ 20 ];
```

Class average example

```
int main()  
{  
    int n;  
    cout << "Enter number of student" << endl;  
    cin >> n;  
    int age[n];  
}
```

Above approach has so many disadvantages

Dynamic memory allocation

In **dynamic memory allocation**, the **heap** is a special region of a computer's memory used for **storing data that is created (allocated) at runtime** , that is, during the execution of a program, not when it is compiled.

Void Pointer A **void pointer** is a **special type of pointer** that can **point to any data type** — int, float, char, double, struct, etc. It is declared using the keyword void.

Declaration Syntax:

```
void *ptr;
```

This means ptr can hold the address of any data type — but you **can't directly dereference** it (access the value) unless you **typecast** it first.

Type casting

```
(int*) ptr;    // *ptr without casting is error
```

Null pointer

Example 1: Basic Use of Void Pointer

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;
    float b = 3.14;
    char c = 'A';

    void *ptr;    // void pointer declaration

    ptr = &a;    // points to int
    cout << "a = " << *(int*)ptr << endl;    // typecast before dereferencing

    ptr = &b;    // points to float
    cout << "b = " << *(float*)ptr << endl;

    ptr = &c;    // points to char
    cout << "c = " << *(char*)ptr << endl;

    return 0;
}
```

Step 1: (int*)ptr

- This part **typecasts** the void* pointer ptr into an int* pointer.
- It **tells the compiler**:

“Hey, treat ptr as a pointer to an integer.”

So now, (int*)ptr is of type **int*** (pointer to int).

Step 2: *(int*)ptr

- The * **dereferences** that int* pointer — meaning it **accesses the value** stored at the memory address.

So if ptr was pointing to an integer with value 10,
then *(int*)ptr gives you the integer 10.

Why Typecasting is Needed?

Since a void pointer has **no associated data type**, the compiler **does not know** how many bytes to read or write. So you must **explicitly tell** it the data type using typecasting:

```
*(int*)ptr    // treat ptr as int pointer
```

```
*(float*)ptr  // treat ptr as float pointer
```

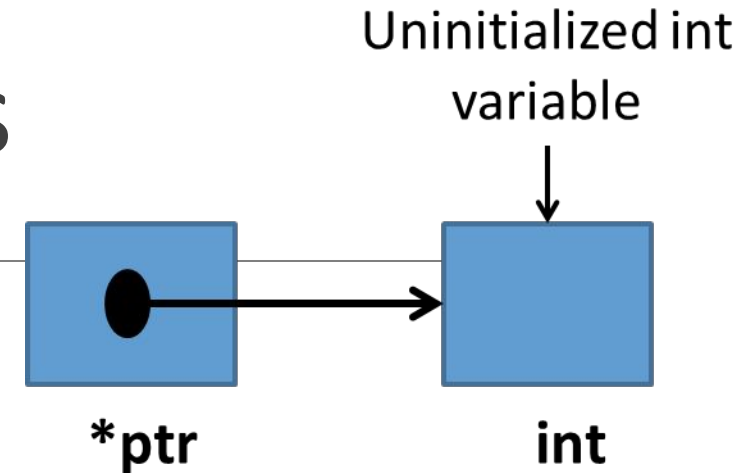

new and delete keywords

New is used to allocate memory dynamically

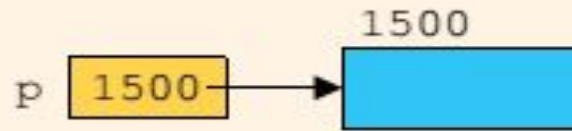
```
int*p; // declares an int pointer  
p =new int; // dynamically allocate memory  
using new  
*p = 10 // assign value to allocated memory
```

delete is used to release the allocated memory

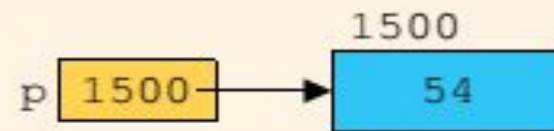
```
delete p;  
delete[] name;  
delete str;
```



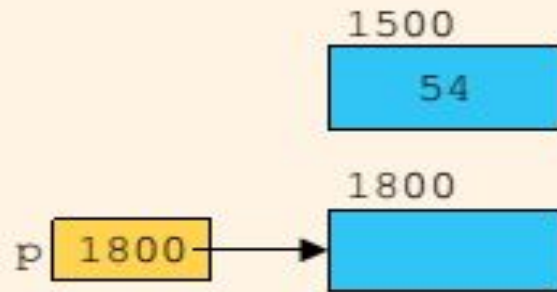
new keyword



(a) `p` after the execution of
`p = new int;`

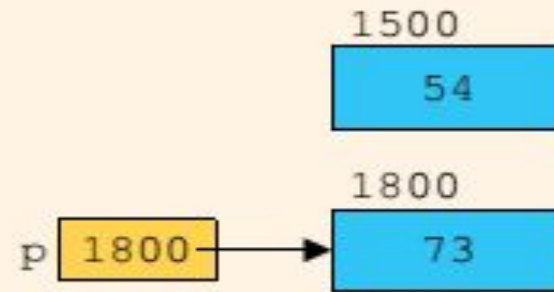


(b) `p` and `*p` after the
execution of `*p = 54;`



(c) `p` after the execution of
`p = new int;`

main



(d) `p` and `*p` after the
execution of `*p = 73;`

Colloc

“calloc” or “contiguous allocation” method in C is used to dynamically allocate the specified number of blocks of memory of the specified type. It is very much similar to malloc() but has two different points and these are:

It initializes each block with a default value ‘0’.

It has two parameters or arguments as compare to malloc().

Syntax:

ptr = (cast-type*)calloc(n, element-size);

here, n is the no. of elements and element-size is the size of each element.

Colloc

Calloc()

```
int* ptr = ( int* ) calloc ( 5, sizeof ( int ) );
```

4 bytes



Allocating memory dynamically

Calloc function

- Arguments
 - Number of elements
 - Size of each element

- Example

```
calloc( 10, 4) ;    // 10 ints x 4 bytes each  
calloc( 10, sizeof ( int ) ) ;  
(int*) calloc ( 1000 , sizeof ( int ) ) ;
```

Is required memory always available ?

Check if memory is allocated or not.

malloc ()

The “malloc” or “memory allocation” method in C is used to dynamically allocate a ***single large block of memory*** with the specified size.

It returns a pointer of type void which can be cast into a pointer of any form.

It doesn't initialize memory at execution time so that it has initialized each block with the default garbage value initially.

Syntax:

ptr = (cast-type*) malloc(byte-size)

For Example:

malloc ()

Malloc()

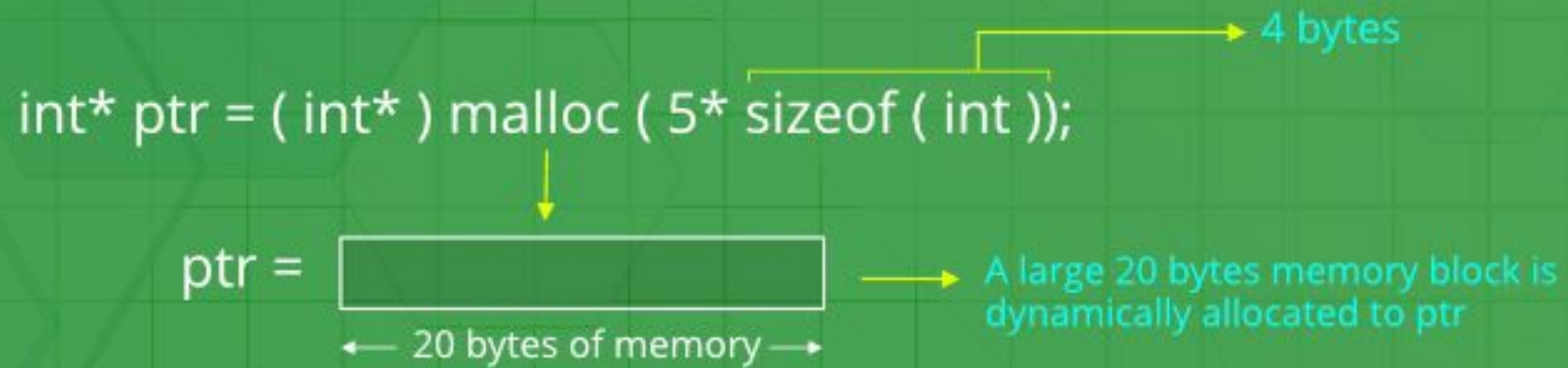
```
int* ptr = ( int* ) malloc ( 5* sizeof ( int ));
```

ptr = 

← 20 bytes of memory →

4 bytes

A large 20 bytes memory block is dynamically allocated to ptr



Increase the allocated memory (realloc)

“realloc” or “re-allocation” method in C is used to dynamically change the memory allocation of a previously allocated memory.

In other words, if the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to dynamically re-allocate memory.

re-allocation of memory maintains the already present value and new blocks will be initialized with the default garbage value.

Syntax:

ptr = realloc(ptr, newSize);

where ptr is reallocated with new size 'newSize'.

Realloc()

```
int* ptr = ( int* ) malloc ( 5* sizeof ( int ));
```

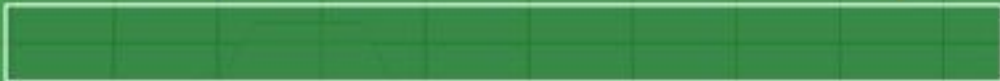
4 bytes

ptr = 

← 20 bytes of memory →

A large 20 bytes memory block is dynamically allocated to ptr

```
ptr = realloc ( ptr, 10* sizeof( int ));
```

ptr = 

← 40 bytes of memory →

The size of ptr is changed from 20 bytes to 40 bytes dynamically

Problems with DMA

Memory leak

- Memory allocated but not released properly

Dangling Pointers

- Pointer pointing to illegal memory

Points to ponder

Using dynamic memory is more efficient than the static memory.

Immediately after a memory allocation call, check whether the memory has allocated successfully.

Whenever possible free the allocated memory in the same function.

Be careful about memory management to prevent memory leakage and dangling pointers.

Before exiting the program, make sure that the allocated memory has freed.